Gestione ed esecuzione di processi mono-tasking, multi-tasking e time sharing

Luca Calzia - 19/10/2026

2 Introduzione e obiettivi

Un processo è un programma in esecuzione. Durante il suo funzionamento può utilizzare la CPU oppure attendere il completamento di un'operazione di input/output. In questo secondo caso non ha bisogno della CPU, che può essere assegnata a un altro processo.

La traccia prevede quattro processi, P1, P2, P3 e P4, che arrivano alla CPU in questo ordine. Ho confrontato i tre metodi osservando il tempo complessivo e la rapidità con cui ciascun processo inizia a essere eseguito.

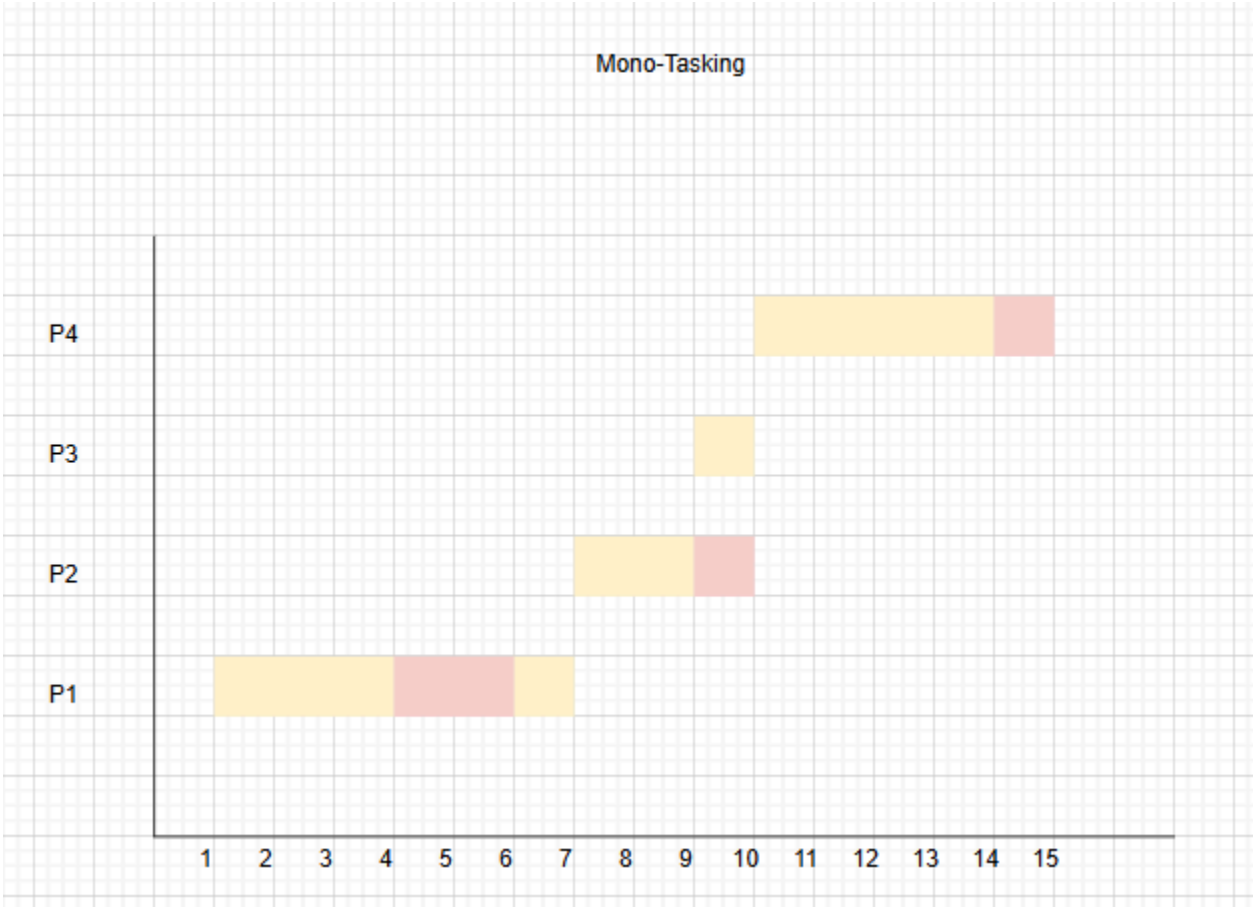
3 Dati e criteri utilizzati

Processo	Tempo di esecuzione	Tempo di attesa	Tempo di esecuzione dopo attesa
P1	3 secondi	2 secondi	1 secondo
P2	2 secondi	1 secondo	-
P3	1 secondi	-	-
P4	4 secondi	1 secondo	-

Ho considerato tutti i processi disponibili all'istante 0, nella coda iniziale P1, P2, P3, P4, perché non sono indicati tempi di arrivo diversi.Non ho conteggiato il tempo necessario ai cambi di contesto e ho incluso nel completamento anche le attese finali di P2 e P4.

4 Mono-tasking

Nel mono-tasking ho eseguito un processo alla volta, rispettando l'ordine di arrivo. Il processo successivo inizia soltanto quando quello precedente ha concluso sia l'esecuzione sia le operazioni di input/output. Durante le attese la CPU rimane quindi inutilizzata.

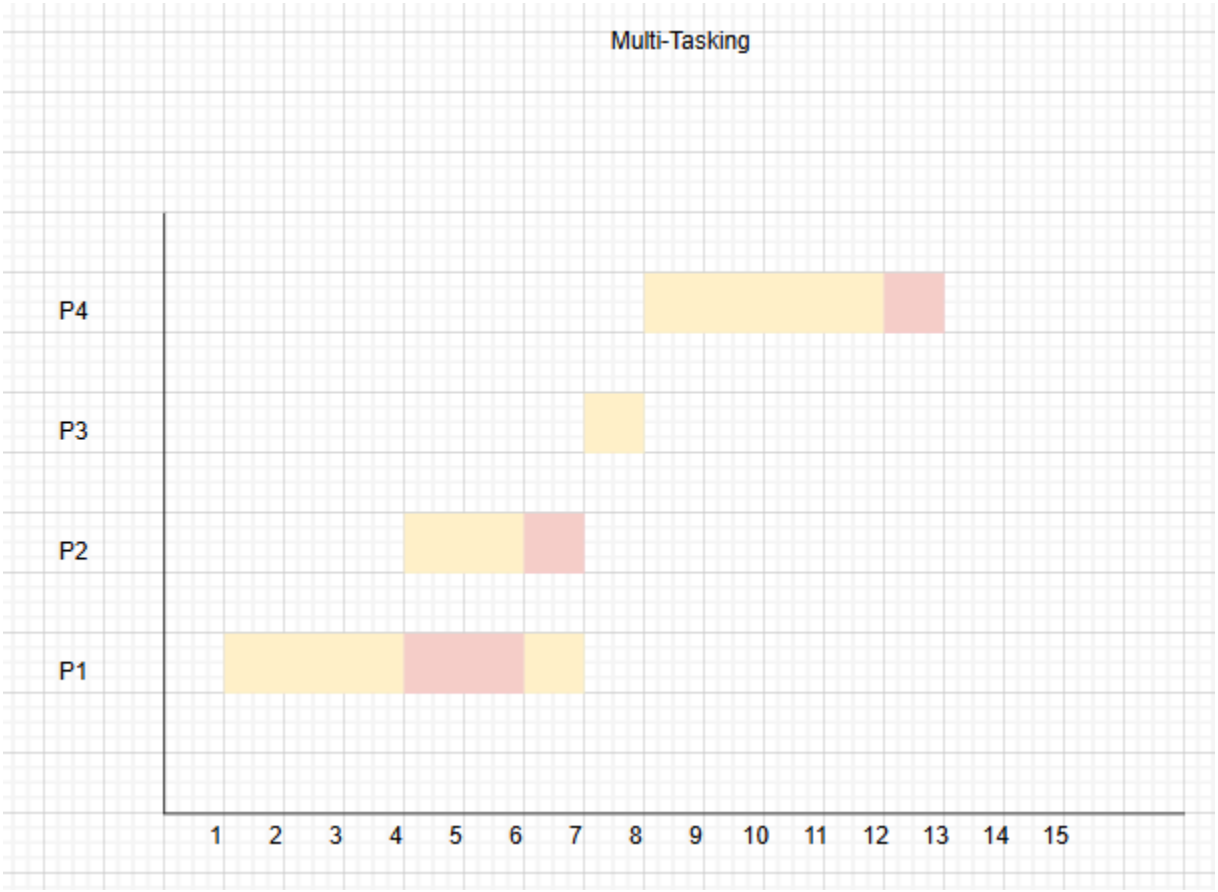


P1 usa la CPU da 0 a 3, attende da 3 a 5 e riprende da 5 a 6. P2 viene eseguito da 6 a 8 e attende fino a 9. P3 lavora da 9 a 10. Infine P4 viene eseguito da 10 a 14 e conclude l'attesa a 15 secondi.

Il tempo complessivo è di 15 secondi: 11 di lavoro della CPU e 4 nei quali la CPU è inattiva. Il metodo è semplice, ma non sfrutta le attese per far avanzare gli altri processi.

5 Multi-tasking

Nel multi-tasking ho assegnato la CPU a un altro processo quando quello in esecuzione entra in attesa. Per questo esempio ho usato una gestione non preemptive: un processo mantiene la CPU fino al termine del proprio tratto di esecuzione o fino a una richiesta di input/output. I processi pronti sono scelti in ordine di ingresso nella coda.

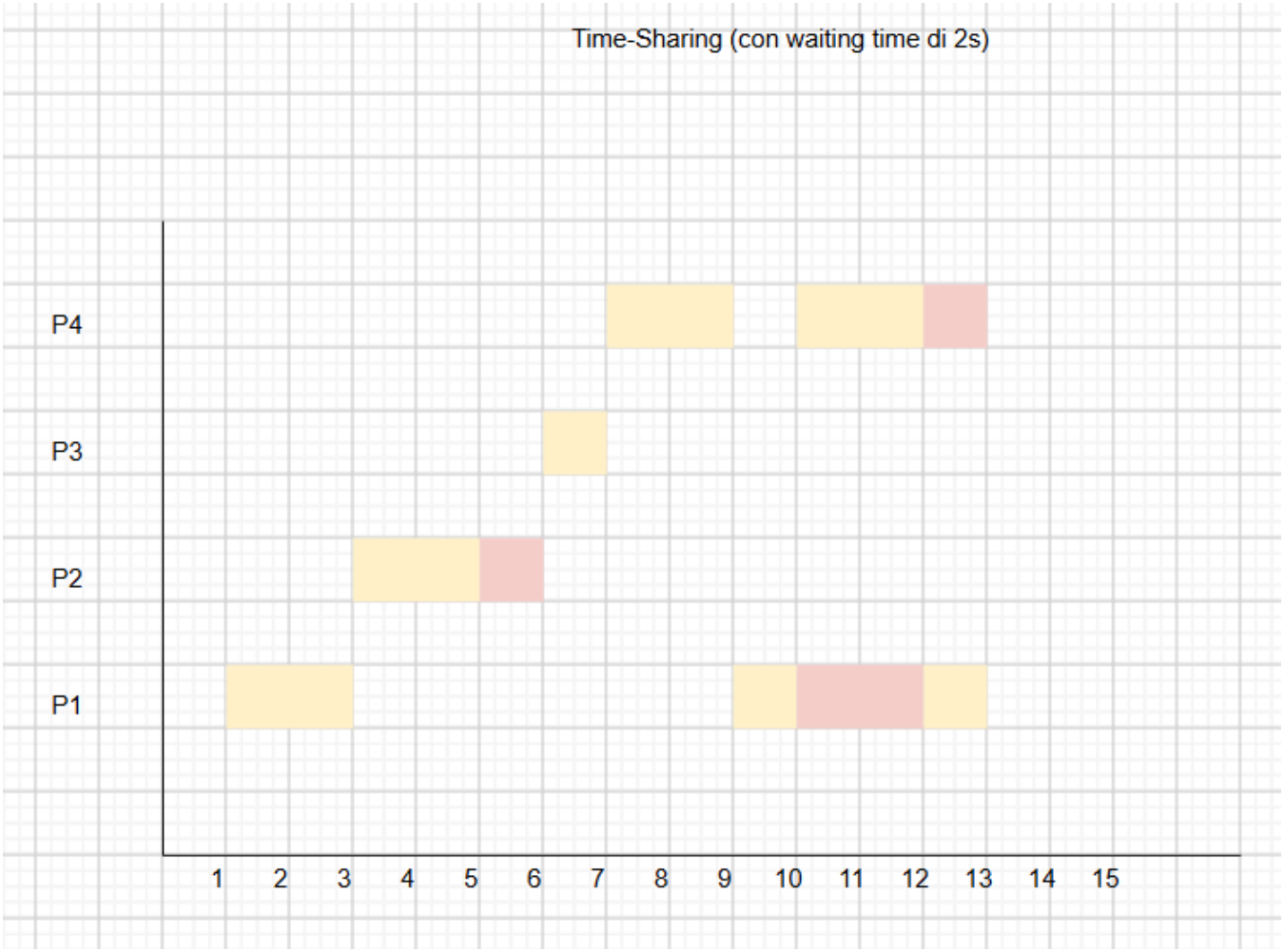


P1 lavora da 0 a 3; durante la sua attesa viene eseguito P2, da 3 a 5. P1 riprende da 5 a 6, mentre P2 completa l'input/output. Seguono P3 da 6 a 7 e P4 da 7 a 11. L'attesa finale di P4 termina a 12 secondi.

Il tempo complessivo scende a 12 secondi. La CPU lavora senza interruzioni da 0 a 11 e resta inattiva soltanto nell'ultimo secondo. Rispetto al mono-tasking vengono risparmiati 3 secondi.

6 Time-sharing con quanto di 2 secondi

Per il time-sharing ho utilizzato il criterio Round Robin con un quanto di tempo di 2 secondi. Ogni processo può utilizzare la CPU per un massimo di 2 secondi consecutivi; se non ha finito e non deve attendere un'operazione di input/output, torna in fondo alla *ready queue*. Anche i processi che completano un'attesa rientrano in fondo alla coda.



7 Confronto dei risultati

Metodo	Tempo complessivo	CPU inattiva	Primo avvio di P4
Mono-tasking	15 s	4 s	10 s
Multi-tasking adottato	12 s	1 s	7 s
Time-sharing (2 s)	11 s	0 s	5 s

Il mono-tasking è il metodo meno efficiente in questo caso, perché blocca l'avanzamento degli altri processi durante le attese. Il multi-tasking sfrutta meglio la CPU. Il time-sharing consente inoltre a tutti i processi di iniziare prima e, con questa sequenza, raggiunge il tempo minimo teorico di 11 secondi, pari al lavoro totale richiesto alla CPU.

8 Conclusioni

L'esercizio mi ha aiutato a distinguere l'attesa di input/output dall'attesa del proprio turno per la CPU. Ho osservato che cambiare processo durante un'attesa permette di ridurre i tempi complessivi e che un quanto di tempo distribuisce l'esecuzione tra più attività.

Con le condizioni adottate, il time-sharing con quanto di 2 secondi ottiene il risultato migliore. Questo non significa che sia sempre il metodo più veloce: il risultato dipende dai tempi dei processi, dalle regole di scelta e dal costo dei cambi di contesto. Il time-sharing è esso stesso una forma di multi-tasking.